## ESCENARIOS DE PRUEBA QA – GRUPO 2

*API y Seguridad | Checklist de seguimiento por prioridad*

| # | Prioridad | ID | Área | Escenario de prueba | Resultado esperado | Seguimiento / Notas | Estado | Evidencias |
|---|---|---|---|---|---|---|---|---|
| 1 | **ALTA** | G2-01 | Login | Iniciar sesión con usuario y contraseña correctos | Permite acceso y crea sesión/token válido | Se realizó una solicitud POST /api/v1/auth/login utilizando credenciales válidas de la cuenta administradora QA. La API respondió HTTP 200 OK, generando un access_token de tipo bearer, con una vigencia declarada de 7200 segundos y los datos correspondientes al usuario autenticado.<br><br>Posteriormente, se utilizó el JWT obtenido para autorizar una solicitud GET /api/v1/me. El endpoint respondió HTTP 200 OK y devolvió correctamente la información del administrador QA (ID 9), incluyendo su rol admin, estado activo y permisos asociados.Se confirmó que el inicio de sesión genera un JWT funcional y permite acceder a un endpoint protegido, completando la validación que había quedado pendiente en la evaluación original. El retest funcional se realizó con la cuenta administradora QA; se conservan como antecedente las seis pruebas de inicio de sesión exitosas documentadas anteriormente. | PASS | Evidencias |
| 2 | **ALTA** | G2-02 | Login | Iniciar sesión con contraseña incorrecta | Rechaza acceso con 401 | Se realizó una solicitud utilizando el correo de la cuenta administradora QA y una contraseña deliberadamente incorrecta.<br><br>La API respondió HTTP 401 Unauthorized, con código de error UNAUTHORIZED y el mensaje genérico "Correo o contraseña incorrectos". La respuesta mantuvo la estructura JSON uniforme del sistema y no incluyó ningún access_token, impidiendo el inicio de sesión.Se confirmó que el sistema continúa rechazando correctamente las contraseñas incorrectas, manteniendo el comportamiento esperado y previamente documentado. | PASS | Evidencias |
| 3 | **ALTA** | G2-05 | Sesión | Usar sesión/token después de cerrar sesión | El acceso debe ser rechazado | Se verificó POST /api/v1/auth/logout con JWT válido de docente, obteniendo HTTP 200 y el mensaje «Sesión cerrada». Posteriormente, se reutilizó el mismo JWT en GET /api/v1/me, obteniendo HTTP 401 con el mensaje «La sesión no existe o ya fue cerrada». Se confirma la revocación efectiva del token tras el cierre de sesión. | PASS | Evidencias |
| 4 | **ALTA** | G2-06 | Autenticación | Acceder a un endpoint protegido sin iniciar sesión | Rechaza acceso con 401 | se verificó la protección de endpoints mediante autenticación JWT utilizando Swagger. Se ejecutó GET /api/v1/me sin autenticación, obteniendo HTTP 401 (UNAUTHORIZED), y posteriormente con un JWT válido de la cuenta docente, obteniendo HTTP 200 con los datos y el rol correspondientes al usuario autenticado. Adicionalmente, se ejecutó GET /api/v1/users sin enviar credenciales ni encabezados de autenticación, obteniendo HTTP 401 con el mensaje "Falta el token de autenticación", sin exponer información protegida.<br><br>Se confirma la corrección del hallazgo original, en el que GET /api/v1/users respondía HTTP 422 por ausencia del encabezado usuario-actual en lugar del HTTP 401 esperado. El escenario cumple el comportamiento esperado dentro del alcance evaluado. | PASS | Evidencias |
| 5 | **ALTA** | G2-08 | Permisos | Usuario autenticado intenta ejecutar una acción para la que no tiene permiso | Rechaza con 403 | se verificó el control de permisos del backend mediante Swagger. Se inició sesión con la cuenta QA docente (rol teacher), obteniendo HTTP 200 y un JWT válido. Posteriormente, se utilizó dicho token para ejecutar GET /api/v1/users, endpoint reservado para administradores.<br><br>La API respondió HTTP 403 (FORBIDDEN), con el mensaje "No tienes permisos necesarios para realizar esta acción", sin devolver información protegida.<br><br>Se confirma que el backend aplica correctamente la restricción de permisos en el endpoint evaluado y que se resolvió el problema de integración JWT que anteriormente impedía completar la prueba, cuando se obtenía HTTP 422.<br><br>El escenario cumple el resultado esperado dentro del alcance probado.. | PASS | Evidencias |
| 6 | **ALTA** | G2-09 | Roles | Docente intenta acceder a funciones exclusivas de administrador | Acceso rechazado | se verificó la restricción de acceso a funciones administrativas utilizando una cuenta autenticada con rol docente (teacher).<br><br>Se inició sesión mediante POST /api/v1/auth/login, obteniendo HTTP 200 y un JWT válido. Posteriormente, se ejecutó GET /api/v1/users utilizando dicho token, con el objetivo de comprobar si un docente podía acceder a una función reservada para administradores.<br><br>La API respondió HTTP 403 (FORBIDDEN), con el mensaje "No tienes permisos necesarios para realizar esta acción", sin exponer información de los usuarios.<br><br>Se confirma la corrección del problema de integración JWT identificado en la evaluación original, cuando el endpoint respondía HTTP 422 e impedía verificar la restricción de acceso.<br><br>El escenario cumple el comportamiento esperado dentro del alcance probado. Se reutilizó la prueba funcional de G2-08, debido a que ambos escenarios evalúan el mismo endpoint y la misma restricción de acceso | PASS | Evidencias |
| 7 | **ALTA** | G2-10 | Roles / Backend | Saltarse el frontend e intentar ejecutar directamente una acción prohibida | El backend rechaza la operación | Se verificó que GET /api/v1/users devuelve HTTP 401 sin JWT, incluso con el encabezado usuario-actual falsificado. Con JWT válido de docente devuelve HTTP 403, tanto sin encabezado como al intentar suplantar a un administrador. Se confirma la corrección de la vulnerabilidad identificada inicialmente en G2-10. | PASS | Evidencias |
| 8 | **ALTA** | G2-15 | Seguridad XSS | Introducir contenido con código/script en campos de texto | El código no se ejecuta en ninguna pantalla | se realizó una prueba de Cross-Site Scripting (XSS) almacenado utilizando el usuario QA con ID 12. Mediante PATCH /api/v1/users/12 se introdujo la cadena <script>alert('QA-XSS')</script> en el campo nombres, obteniendo HTTP 200. Posteriormente, GET /api/v1/users/12 confirmó que el contenido permanecía almacenado.<br><br>Se revisó el archivo frontend-login/script.js y se identificó que el nombre del usuario se muestra mediante alert() después de un inicio de sesión exitoso. Al iniciar sesión con la cuenta de prueba, el navegador mostró la cadena introducida como texto literal dentro del mensaje de bienvenida, sin observarse la ejecución del script.<br><br>Finalmente, se restauró el campo nombres a su valor original "QA" mediante PATCH, obteniendo HTTP 200. | PASS (caso evaluado) | Evidencias |

| | | | | | | | | |
|---|---|---|---|---|---|---|---|---|
| 9 | **ALTA** | G2-16 | Seguridad SQL | Probar entradas orientadas a SQL Injection | La entrada no altera consultas ni la base de datos | se realizó una prueba de SQL Injection en POST /api/v1/auth/login, introduciendo la cadena ' OR '1'='1 en el campo correo. La API respondió HTTP 401 (Unauthorized), con el código UNAUTHORIZED y el mensaje «Correo o contraseña incorrectos», sin generar un token de acceso ni permitir la autenticación.<br><br>Posteriormente, se ejecutó GET /api/v1/users con credenciales administrativas válidas, obteniendo HTTP 200 y recuperando correctamente los registros de usuarios. Adicionalmente, se revisó el código de g2_api/routers/auth.py, confirmando que la búsqueda del usuario utiliza filtros de SQLAlchemy mediante Usuario.correo == datos.correo, sin concatenación directa de la entrada en sentencias SQL. También se verificó el uso de filtros de SQLAlchemy en la consulta de sesiones.<br><br>El escenario se considera PASS (caso evaluado), ya que el intento de inyección no permitió evadir la autenticación y las consultas posteriores funcionaron correctamente. No se realizó una comparación de los registros antes y después de la prueba, por lo que no puede confirmarse de manera exhaustiva la ausencia de modificaciones en la base de datos. El resultado se limita al vector de inyección y a los componentes revisados. | PASS (caso evaluado) | Evidencias |
| 10 | **ALTA** | G2-17 | Autorización | Cambiar un identificador para intentar acceder a recursos de otro usuario | El servidor verifica autorización y bloquea el acceso | se evaluó el control de autorización sobre recursos de otros usuarios mediante el endpoint PATCH /api/v1/users/{usuario_id}, utilizando Swagger y autenticación JWT.<br><br>Inicialmente, se ejecutó una solicitud PATCH /api/v1/users/999999 con un JWT válido de la cuenta QA docente (ID 11), obteniendo HTTP 403 (FORBIDDEN), en lugar de permitir el procesamiento de la operación sin verificar los permisos.<br><br>Posteriormente, se realizó una prueba de acceso horizontal y vertical utilizando el mismo JWT docente para intentar modificar la cuenta QA administradora (ID 9) mediante PATCH /api/v1/users/9, enviando el campo nombres con el valor "Prueba QA". La API respondió HTTP 403 (FORBIDDEN), con el mensaje "No tienes permisos necesarios para realizar esta acción".<br><br>Finalmente, se inició sesión como administrador y se consultó GET /api/v1/me, obteniendo HTTP 200. Se verificó que el nombre del administrador permanecía como "QA", confirmando que el intento de modificación no alteró sus datos.<br><br>Se confirma la corrección del hallazgo original relacionado con el procesamiento de solicitudes sin autenticación JWT en la ruta administrativa evaluada. El backend rechazó los intentos de modificación no autorizados y preservó la integridad de los datos del usuario objetivo. | PASS | Evidencias |
| 11 | **ALTA** | G2-19 | Archivos | Intentar subir un tipo de archivo no permitido | Archivo rechazado | Se verificó que POST /api/v1/files rechaza archivos no permitidos. Un archivo TXT original obtuvo HTTP 400. Posteriormente, se modificó únicamente su extensión a .pdf, manteniendo su contenido de texto plano; la API volvió a responder HTTP 400 con código BAD_REQUEST. Se confirma el rechazo de ambos casos y la validación del contenido en el caso probado. | PASS | Evidencias |
| 12 | **ALTA** | G2-20 | Archivos | Intentar subir un archivo que exceda el tamaño máximo | Archivo rechazado | Durante el retest del 23/10/2026 se confirmó que POST /api/v1/files implementa una restricción de tamaño máximo de 5 MB. Se verificó en el código fuente la constante MAX_SIZE = 5 * 1024 * 1024 y su validación. Al intentar cargar un archivo de 6 MB, la API respondió HTTP 400 Bad Request con el mensaje "El archivo supera el límite de 5 MB". El resultado coincide con el comportamiento esperado. Escenario aprobado. | PASS | Evidencias |
| 13 | **ALTA** | G2-21 | Archivos / Permisos | Intentar descargar o acceder al archivo de otro usuario sin autorización | Acceso rechazado | Se verificó la corrección de seguridad en el endpoint GET /api/v1/files/{id} sobre el despliegue correspondiente al commit 5bc6452.<br><br>Se creó un archivo PDF de prueba con ID 3 y se validaron los permisos de acceso mediante tres usuarios:<br><br>Propietario (docente1): HTTP 200 — descarga autorizada.<br>Usuario ajeno (docente2): HTTP 403 — acceso denegado correctamente.<br>Administrador (admin): HTTP 200 — descarga autorizada.<br>El endpoint cumple con la restricción de descarga exclusiva para el propietario o un administrador. | PASS | Evidencias |
| 14 | **ALTA** | G2-22 | Archivos / Seguridad | Intentar manipular el nombre o ruta del archivo | No permite acceder ni escribir fuera del almacenamiento autorizado | Durante el retest se realizaron intentos de acceso mediante rutas manipuladas (../main.py y C:\Windows\win.ini), obteniendo HTTP 404 sin exposición de archivos internos. Adicionalmente, se revisó files.py y se verificó que los archivos se almacenan en la carpeta uploads con un nombre interno generado mediante UUID y extensión controlada por el tipo MIME permitido, sin utilizar directamente el nombre proporcionado por el usuario. El escenario se considera aprobado dentro del alcance evaluado. | PASS | Evidencias |
| 15 | **ALTA** | G2-23 | Errores / Seguridad | Provocar errores para buscar información interna | No muestra SQL, rutas internas, stack traces ni secretos | se realizaron dos pruebas de manejo de errores en POST /api/v1/auth/login para verificar que las respuestas no expongan información interna sensible.<br><br>En la primera prueba se envió un JSON mal formado, obteniendo HTTP 422 (Unprocessable Content), con el código VALIDATION_ERROR y un detalle controlado de tipo json_invalid. En la segunda prueba se enviaron valores numéricos en los campos correo y password, obteniendo nuevamente HTTP 422, con errores string_type para ambos campos.<br><br>En las respuestas evaluadas no se observaron consultas SQL, rutas internas del servidor, stack traces, contraseñas, tokens ni otros secretos. La API devolvió mensajes de error estructurados mediante los campos error, code, message y details. | PASS (casos evaluados) | Evidencia |
| 16 | **ALTA** | G2-24 | Información sensible | Revisar respuestas de API buscando información sensible | No expone contraseñas, secretos ni información sensible innecesaria | Al realizar una solicitud autenticada con un JWT de administrador, la API respondió HTTP 200 y devolvió una lista de usuarios con los campos id, correo, nombres, apellidos, activo y roles. No se observaron contraseñas, hashes, tokens ni claves secretas en los registros revisados.<br><br>Adicionalmente, se inspeccionó la función diccionario() del archivo g2_api/routers/users.py, confirmando que construye las respuestas únicamente con los seis campos mencionados y excluye información sensible como hash_contrasena, password, access_token y refresh_token.<br><br>Finalmente, se realizó una solicitud GET /api/v1/users | PASS | Evidencias |

| | | | | | | | | |
|---|---|---|---|---|---|---|---|---|
| 17 | **ALTA** | G2-35 | Configuración | Revisar repositorio y configuración | No contiene contraseñas ni variables sensibles | Se verificó que el archivo .gitignore contiene reglas para excluir archivos .env, entornos virtuales, bases de datos locales, archivos de registro, directorios de carga y otros archivos potencialmente sensibles. Asimismo, se revisó .env.example y se confirmó que utiliza valores de ejemplo para DATABASE_URL y SECRET_KEY, sin identificar credenciales reales en la plantilla.<br><br>Posteriormente, se inspeccionó en GitHub la rama grupo-2 del repositorio UDV_Proyecto, específicamente la carpeta grupos/grupo-2-api-seguridad. No se observaron archivos .env, bases de datos locales ni directorios de carga publicados.<br><br>Adicionalmente, se utilizó el buscador de archivos de GitHub para localizar archivos relacionados con .env en la rama evaluada. La búsqueda mostró únicamente dos archivos .env.example, uno en la raíz del repositorio y otro en la carpeta del Grupo 2. | PASS | Evidencias |
| 18 | **ALTA** | G2-37 | Autenticación / Seguridad | Usar un token vencido para acceder a un endpoint protegido | Devuelve 401 | Se revisó la función verificar_token() en g2_api/seguridad.py, confirmando que utiliza jwt. decode() para validar la firma y expiración del token y rechaza los tokens inválidos o expirados.<br><br>Asimismo, se comprobó su integración con obtener_sesion_actual() en el módulo de autenticación. Posteriormente, se generó un JWT de prueba correctamente firmado, con una fecha de expiración anterior a la actual. La validación local rechazó el token y, al utilizarlo para acceder al endpoint protegido GET /api/v1/me, la API respondió HTTP 401 Unauthorized, con código UNAUTHORIZED y mensaje "Token inválido o vencido". | PASS | Evidencias |
| 19 | **ALTA** | G2-38 | Autenticación / Seguridad | Usar un token modificado manualmente para acceder a un endpoint protegido | Devuelve 401 | Inicialmente, se revisó la prueba automatizada test_token_alterado_no_es_valido, ubicada en tests/test_auth.py, y se ejecutó mediante pytest, obteniendo un resultado satisfactorio (1 passed).<br><br>Posteriormente, se generó un JWT firmado con la configuración local del proyecto y se modificó manualmente un carácter de su firma sin volver a firmarlo. Al utilizar el token alterado para acceder al endpoint protegido GET /api/v1/me, la API respondió HTTP 401 Unauthorized, con código de error UNAUTHORIZED y mensaje "Token inválido o vencido". Se confirmó que el endpoint evaluado rechaza correctamente un JWT cuya firma ha sido modificada, resolviendo el bloqueo identificado en la evaluación original. La comprobación funcional se realizó sobre GET /api/v1/me. | PASS — JWT alterado rechazado correctamente | Evidencias |
| 20 | **MEDIA** | G2-03 | Login | Iniciar sesión con usuario inexistente | Acceso rechazado sin revelar información sensible | se realizó una solicitud POST /api/v1/auth/login mediante Swagger, utilizando un correo no registrado y una contraseña de prueba. La API respondió HTTP 401 (Unauthorized), con el código UNAUTHORIZED y el mensaje genérico «Correo o contraseña incorrectos», sin generar un token de acceso ni revelar información sobre la existencia del usuario.<br><br>Posteriormente, se repitió la prueba desde el formulario de inicio de sesión del frontend en http://localhost:5500, utilizando las mismas credenciales. El sistema mostró el mensaje «Correo o contraseña incorrectos» y rechazó el inicio de sesión.Se confirmó que tanto la API como el frontend rechazan el acceso con un usuario inexistente sin revelar información sensible sobre la existencia de la cuenta. | PASS | Evidencias |
| 21 | **MEDIA** | G2-04 | Login / Validación | Enviar usuario y contraseña vacíos | Solicitud rechazada mediante validación | Se realizaron pruebas de inicio de sesión con campos vacíos tanto en la API como en el frontend. En la API, al enviar correo y contraseña vacíos, se obtuvo HTTP 401 (Unauthorized), rechazando el acceso. Posteriormente, desde http://localhost:5500, se comprobó que el formulario identifica los campos obligatorios: al dejar Usuario vacío, solicita completarlo; al ingresar un correo y dejar Contraseña vacía, muestra la misma validación. En ambos casos, el navegador impide enviar el formulario. Se cumple el resultado esperado de rechazar el inicio de sesión con campos vacíos. Como observación, la API devuelve 401 en lugar de un error de validación 422. | PASS | Evidencias |
| 22 | **MEDIA** | G2-07 | Identidad | Consultar /me con un usuario autenticado | Devuelve usuario, roles y permisos correspondientes | Se verificó el funcionamiento de GET /api/v1/me. Sin token JWT devuelve HTTP 401. Con JWT válido de docente devuelve la identidad correcta (ID 11), correo, estado activo, rol teacher y permisos asociados. Se confirma la implementación del endpoint anteriormente no disponible. | PASS | Evidencia |
| 23 | **MEDIA** | G2-11 | Usuarios | Consultar un usuario existente | Devuelve la información permitida correctamente | Se realizó la consulta sin autenticación JWT válida, obteniendo HTTP 401 (Unauthorized), lo que confirmó que el endpoint exige autenticación.Posteriormente, se repitió la solicitud utilizando un token JWT de administrador. La API respondió HTTP 200 (OK) y devolvió correctamente la información del usuario solicitado, incluyendo únicamente los campos id, correo, nombres, apellidos, activo y roles, sin exponer contraseñas, hashes ni tokens.<br><br>Adicionalmente, se realizó la misma consulta utilizando un token JWT correspondiente a un usuario con rol docente. La API respondió HTTP 403 (Forbidden), rechazando el acceso por falta de permisos administrativos. En comparación con la evaluación original, se confirmó que actualmente existe el endpoint de consulta individual y que se aplican controles de autenticación y autorización basados en JWT y roles. | PASS — Retest satis | Evidencias |
| 24 | **MEDIA** | G2-12 | Usuarios | Consultar un ID de usuario inexistente | Responde 404 | Se verificó GET /api/v1/users/{usuario_id} con JWT de administrador. La consulta del usuario existente (ID 11) devolvió HTTP 200 con los datos correctos, sin exponer contraseñas ni hashes. La consulta del ID inexistente 999999 devolvió HTTP 404, código NOT_FOUND y mensaje descriptivo. Se confirma la implementación y el manejo adecuado de usuarios inexistentes. | PASS | Evidencias |
| 25 | **MEDIA** | G2-13 | Validación | Enviar datos obligatorios vacíos | Responde 422 indicando los campos inválidos | Se ejecutó una solicitud incluyendo únicamente el campo correo y omitiendo el campo obligatorio password. La API respondió HTTP 422 (Unprocessable Content), rechazando la solicitud por incumplimiento de los requisitos de validación.<br><br>Posteriormente, se realizó una segunda solicitud incluyendo únicamente el campo password y omitiendo correo. La API respondió HTTP 422, identificando correctamente el campo correo mediante el mensaje "Field required" y el tipo de error "missing".<br><br>Los resultados confirman que la API rechaza solicitudes incompletas e identifica los campos obligatorios ausentes. Se mantiene el comportamiento esperado de validación observado en la evaluación original. | PASS | Evidencias |

| | | | | | | | | |
|---|---|---|---|---|---|---|---|---|
| 26 | **MEDIA** | G2-14 | Validación | Enviar datos con formato incorrecto | No procesa ni almacena información inválida | Se realizó una solicitud enviando el campo correo con un valor numérico (12345) en lugar de una cadena de texto. La API respondió HTTP 422 (Unprocessable Content), rechazando la solicitud con el mensaje "Input should be a valid string". Posteriormente, se ejecutó una segunda solicitud enviando el campo password con un valor numérico (12345678). La API respondió nuevamente HTTP 422, con código VALIDATION_ERROR, identificando el campo password mediante el mensaje "Input should be a valid string" y el tipo de error string_type.<br><br>Ambas pruebas confirmaron que el backend identifica y rechaza tipos de datos incompatibles antes de completar el proceso de autenticación. No se verificó directamente la persistencia de datos en la base de datos, por lo que la conclusión se limita al rechazo de las solicitudes inválidas. | PASS | Evidencias |
| 27 | **MEDIA** | G2-18 | Archivos | Subir un tipo de archivo permitido | Archivo aceptado y devuelve file_id | Se verificó la carga de un archivo PDF válido mediante POST /api/v1/files, obteniendo HTTP 200 y un file_id único. Posteriormente, se recuperó el archivo mediante GET /api/v1/files/{id}, obteniendo HTTP 200 y confirmando que el PDF descargado corresponde al original. Se observó una inconsistencia en Swagger: el identificador se envía como parámetro query (file_id) en lugar de sustituir el parámetro de ruta {id}. Se recomienda revisar el contrato del endpoint. | PASS | Evidencias |
| 28 | **MEDIA** | G2-25 | API / HTTP | Realizar correctamente una solicitud de creación | Utiliza el código HTTP correspondiente, por ejemplo 201 | Se intentó crear un usuario utilizando un token JWT correspondiente al rol docente. La API respondió HTTP 403 (Forbidden), rechazando correctamente la operación por falta de permisos administrativos.<br><br>Posteriormente, se repitió la solicitud utilizando un token JWT de administrador y datos válidos de un usuario ficticio. El backend respondió HTTP 201 (Created), confirmando la creación del usuario y devolviendo su información con los campos permitidos, sin exponer contraseñas, hashes ni tokens.<br><br>Finalmente, se realizó una solicitud de creación sin presentar un token JWT. La API respondió HTTP 401 (Unauthorized), impidiendo la operación sin autenticación.<br><br>En comparación con la evaluación original, se confirmó que la creación de usuarios continúa utilizando correctamente HTTP 201 y que el acceso al endpoint ahora requiere autenticación JWT y permisos administrativos. La vulnerabilidad previamente observada, que permitía realizar operaciones administrativas sin JWT mediante el encabezado usuario-actual, no se reprodujo durante el retest. | PASS | Evidencias |
| 29 | **MEDIA** | G2-26 | API / HTTP | Solicitar un recurso inexistente | Devuelve 404 | Se verificó el comportamiento de la API al solicitar un recurso inexistente mediante los endpoints GET y PATCH /api/v1/users/{usuario_id}, utilizando el ID 999999 y autenticación JWT de administrador. Inicialmente, se ejecutó una solicitud GET /api/v1/users/999999. El backend respondió HTTP 404 (Not Found), con el mensaje "Usuario no encontrado", confirmando que identifica correctamente la ausencia del recurso solicitado.<br><br>Posteriormente, se realizó una solicitud PATCH /api/v1/users/999999 enviando un cuerpo JSON vacío ({}). La API respondió nuevamente HTTP 404 (Not Found), rechazando la modificación del usuario inexistente.En comparación con la evaluación original, se confirmó que actualmente está disponible el endpoint GET para consultar usuarios individualmente. Anteriormente, esta solicitud devolvía HTTP 405 (Method Not Allowed), lo que impedía completar la prueba mediante GET. Durante el retest, ambos métodos respondieron con el código HTTP esperado. | PASS | Evidencias |
| 30 | **MEDIA** | G2-27 | API / Estados | Intentar crear un usuario con un correo ya registrado | Devuelve 409 y el usuario duplicado no se crea | Se confirmó mediante GET /api/v1/me que la solicitud se realizó con una cuenta administradora autorizada. Posteriormente, se ejecutó POST /api/v1/users intentando crear un usuario con el correo previamente registrado qa.revisor@qa.udv.edu.gt. La API respondió HTTP 409 (Conflict), con código de error CONFLICT y mensaje "El correo ya está registrado".<br><br>Adicionalmente, se realizó una consulta de solo lectura a MySQL sobre la tabla usuario, confirmando que existe únicamente un registro asociado al correo evaluado (ID 12). Por tanto, se verificó que la API rechazó la solicitud y no generó un usuario duplicado. | PASS | Evidencias |
| 31 | **MEDIA** | G2-28 | API / Validación | Provocar un error de validación | Devuelve 422 | Se ejecutó una solicitud POST /api/v1/users con una cuenta administradora autorizada, enviando intencionalmente un valor numérico (12345) en el campo correo, que requiere una cadena de texto.<br><br>La API respondió HTTP 422 (Unprocessable Content), con código VALIDATION_ERROR y mensaje "Datos de entrada inválidos". En el campo details.campos se identificó correctamente el atributo correo, con el mensaje "Input should be a valid string" y tipo de error string_type.<br><br>Se confirmó que la API rechaza los datos de entrada inválidos mediante una respuesta JSON estructurada, sin completar la creación del usuario. El escenario cumple el resultado esperado con el mecanismo actual de autenticación JWT. | PASS | Evidencias |
| 32 | **MEDIA** | G2-29 | API / JSON | Revisar la estructura de las respuestas | Respeta el formato JSON acordado | La API respondió HTTP 422 (Unprocessable Content), con encabezado Content-Type: application/json y una estructura de error uniforme que contiene los campos error, code, message y details. Se obtuvo el código VALIDATION_ERROR, el mensaje "Datos de entrada inválidos" y el detalle "JSON decode error", con tipo json_invalid.<br><br>Se confirmó que el backend rechaza solicitudes con sintaxis JSON inválida y devuelve una respuesta estructurada sin exponer información sensible. El código principal de error cambió respecto a la prueba original, de JSON_INVALID a VALIDATION_ERROR, manteniendo la identificación específica del error mediante json_invalid. | PASS | Evidencia |
| 33 | **MEDIA** | G2-30 | API / Errores | Revisar la estructura de las respuestas de error | Mantiene el formato común definido para errores | se realizaron pruebas funcionales en Swagger sobre los códigos HTTP 400 (BAD_REQUEST), 401 (UNAUTHORIZED), 403 (FORBIDDEN), 404 (NOT_FOUND) y 422 (VALIDATION_ERROR). Se confirmó que todas las respuestas evaluadas mantienen una estructura JSON uniforme con los campos error, code, message y details. Adicionalmente, se revisaron las funciones _payload() y error_response() del manejador de errores, verificando que utilizan el formato común actualizado. El escenario cumple el comportamiento esperado dentro del alcance probado; no se evaluaron todos los posibles errores del sistema. | PASS — Errores evaluados | Evidencias |

| | | | | | | | | |
|---|---|---|---|---|---|---|---|---|
| 34 | **MEDIA** | G2-33 | OpenAPI | Comparar endpoints documentados contra los endpoints reales | OpenAPI coincide con el comportamiento real del sistema | Se realizó el retest mediante la revisión de la documentación OpenAPI publicada en Swagger y su comparación con las respuestas reales de los endpoints POST /api/v1/users, GET /api/v1/me y GET /api/v1/users en Railway.<br><br>En POST /api/v1/users se confirmó que Swagger continúa omitiendo HTTP 409 (Conflict), respuesta comprobada previamente en G2-27 mediante una prueba local. Asimismo, el ejemplo HTTP 201 sigue mostrando un valor genérico "string" en lugar de representar la estructura JSON de creación de usuarios. Se verificó también que HTTP 422 devuelve correctamente VALIDATION_ERROR, con details como objeto y una llave por cada campo inválido, cumpliendo lo solicitado por el PM. Sin embargo, Swagger continúa mostrando el esquema genérico de FastAPI, que no coincide con la respuesta real.<br><br>En GET /api/v1/me se confirmó que HTTP 401 está documentado correctamente. No obstante, el ejemplo HTTP 200 continúa mostrando "string", mientras que Railway devuelve un objeto JSON con los datos, roles y permisos del usuario autenticado.<br><br>Finalmente, en GET /api/v1/users se comprobó que Swagger presenta un ejemplo HTTP 200 de tipo "string", mientras que la API devuelve un objeto JSON con los campos page, per_page, total y data. Adicionalmente, se ejecutó una solicitud utilizando una cuenta docente, obteniendo HTTP 403 (Forbidden), con código FORBIDDEN y un mensaje de permisos insuficientes. Swagger identificó esta respuesta como Undocumented, confirmando que HTTP 403 continúa ausente de la documentación.<br><br>Resultado: Se confirma que los endpoints evaluados presentan respuestas funcionales coherentes con las pruebas ejecutadas, pero persisten discrepancias entre el comportamiento real de la API y los códigos HTTP y esquemas publicados en OpenAPI. Se requiere actualizar la documentación Swagger para cumplir el criterio de aceptación del escenario. No se identificaron nuevos defectos funcionales en las solicitudes evaluadas. | FAIL | Evidencias |
| 35 | **BAJA** | G2-31 | Paginación | Solicitar un listado con varias páginas | page, per_page, total y data funcionan correctamente | Se realizaron pruebas funcionales para verificar el comportamiento de los parámetros page y per_page del endpoint de consulta de usuarios, utilizando solicitudes HTTP directamente contra el servidor desplegado en Railway.<br><br>Se comprobaron satisfactoriamente las siguientes condiciones:<br>-Rechazo de valores cero, negativos, decimales, no numéricos y vacíos, mediante respuestas HTTP 422 con código VALIDATION_ERROR.<br>-Validación del rango permitido para per_page, comprendido entre 1 y 100 registros.<br>-Aceptación de los valores mínimos y máximos permitidos, mediante respuestas HTTP 200.<br>-Funcionamiento correcto de la paginación en la primera, segunda y última página.<br>-Respuesta HTTP 200 con lista vacía al solicitar páginas fuera del rango disponible.<br>-Aplicación automática de los valores predeterminados page=1 y per_page=20 cuando se omiten los parámetros correspondientes.<br>-Identificación simultánea de errores cuando ambos parámetros contienen valores inválidos.<br><br>Durante las pruebas, el servidor reportó un total de 16 usuarios. Se verificó que las solicitudes válidas devolvieran la cantidad correspondiente de registros y que las solicitudes inválidas fueran rechazadas con mensajes específicos de validación. | PASS | Evidencias |
| 36 | **BAJA** | G2-32 | Fechas | Revisar las fechas devueltas por la API | Utilizan ISO 8601 y la zona horaria acordada | En la evaluación original se obtuvo un resultado PASS al identificar el campo `creado_en` en formato ISO 8601 mediante `GET /api/v1/users`. Durante un retest posterior se observó que la API no devolvía campos de fecha, por lo que el escenario quedó pendiente de validación.<br><br>El 05/11/2026 se realizó una nueva evaluación directamente contra el servidor desplegado en Railway, comprobando que el endpoint `GET /api/v1/users` vuelve a incluir los campos `creado_en` y `actualizado_en`.<br><br>Se ejecutaron ocho pruebas funcionales con resultados satisfactorios. Se verificó la presencia de los campos de fecha, su formato ISO 8601 y su consistencia en distintas páginas de resultados. Adicionalmente, mediante PowerShell se analizaron los 16 usuarios devueltos por la API, validando un total de 32 fechas.<br><br>Los resultados confirmaron que las 32 fechas cumplen el patrón ISO 8601 evaluado, pueden interpretarse correctamente como fechas válidas y contienen el indicador de zona horaria UTC (`Z`). Asimismo, se verificó que los 16 usuarios presentan fechas de actualización iguales o posteriores a sus respectivas fechas de creación.<br><br>No se identificaron errores de formato, valores de fecha inválidos ni inconsistencias cronológicas en los registros evaluados.<br><br>**Resultado: 8/8 pruebas PASS.** Se confirma satisfactoriamente el formato ISO 8601 y la representación UTC de las fechas devueltas por `GET /api/v1/users` | PASS | Evidencias |

| | | | | | | | | |
|---|---|---|---|---|---|---|---|---|
| 37 | **BAJA** | G2-34 | Login / Seguridad | Realizar 5 intentos fallidos consecutivos de login | El usuario queda bloqueado durante 15 minutos | se evaluó el mecanismo de bloqueo temporal por intentos fallidos de inicio de sesión mediante Swagger, utilizando la cuenta QA qa.revisor@qa.udv.edu.gt (rol reviewer) Inicialmente, se verificó que la cuenta podía iniciar sesión correctamente mediante POST /api/v1/auth/login, obteniendo HTTP 200.<br><br>Posteriormente, se realizaron cinco intentos consecutivos de inicio de sesión con una contraseña incorrecta. Los primeros cuatro intentos devolvieron HTTP 401 (UNAUTHORIZED), mientras que el quinto devolvió HTTP 429 (TOO_MANY_REQUESTS), con el mensaje "Cuenta bloqueada por intentos fallidos. Intenta de nuevo en 15 minutos".<br><br>Durante el período de bloqueo, se intentó iniciar sesión utilizando la contraseña correcta. La API respondió nuevamente HTTP 429, indicando que la cuenta continuaba bloqueada y que faltaban aproximadamente 13 minutos para permitir un nuevo intento.<br><br>Finalmente, después de transcurrido el período de bloqueo, se realizó un nuevo inicio de sesión con las credenciales correctas, obteniendo HTTP 200 y un nuevo access_token. | PASS | Evidencias |
| 38 | **BAJA** | G2-36 | Auditoría | Realizar una acción sensible | El acceso o acción queda registrado según corresponda | se verificó funcionalmente el registro de auditoría mediante Swagger y consultas de solo lectura a MySQL.Inicialmente, se ejecutó POST /api/v1/auth/login con un correo inexistente, obteniendo HTTP 401. Posteriormente, se confirmó en la tabla auditoria la creación del evento LOGIN_FALLIDO (ID 74), con registro_id = 0, actor_id = NULL y el correo utilizado en el campo detalle. Esto confirma la corrección del problema reportado previamente, relacionado con la restricción NOT NULL de registro_id.<br><br>Adicionalmente, se revisó la integración de auditoría en el código y se consultaron los eventos existentes en MySQL. Para validar una acción sensible, se actualizó temporalmente el nombre del usuario QA ID 12 mediante PATCH /api/v1/users/12, utilizando la cuenta administradora ID 9. La operación devolvió HTTP 200 y generó el registro de auditoría ID 75, con acción update, tabla usuario, registro_id = 12 y actor_id = 9. Finalmente, se restauró el nombre original del usuario de prueba mediante PATCH, obteniendo HTTP 200.<br><br>Se confirma que los intentos fallidos de autenticación con correos inexistentes y las actualizaciones de usuarios evaluadas quedan registrados correctamente en la base de datos. El hallazgo original queda corregido dentro del alcance probado; no se evaluaron todas las posibles acciones sensibles del sistema. | PASS | Evidencias |

*Prioridad ALTA: validar primero por impacto en seguridad y funcionamiento.*
*Prioridad MEDIA: validar contrato, comportamiento e integración de la API.*
*Prioridad BAJA: validaciones complementarias que igualmente deben revisarse antes del cierre.*